Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «УрФУ имени первого Президента России Б.Н. Ельцина»
Кафедра «школа бакалавриата (школа)»

Оценка работы _____
Руководитель от УрФУ Корнякова Е.М.

Тема задания на практику

Разработка интеллектуальной системы личной памяти «Memory Hub»

ОТЧЕТ
Вид практики: Производственная практика
Тип практики: Производственная практика, преддипломная

Руководитель практики от предприятия (организации) Исмагилов Р. Т. _____
ФИО руководителя Подпись

Студент: Гомаа Х. Н. А. З _____
ФИО студента Подпись

Специальность (направление подготовки) 09.03.04 Программная инженерия

Группа РИ-420947

Екатеринбург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ОПИСАНИЕ РАБОТЫ, ВЫПОЛНЕННОЙ В ПЕРИОД ПРАКТИКИ 5

1.1 Характеристика предметной области и постановка задачи 5

1.2 Функциональные требования и пользовательские сценарии 8

1.3 Календарный план и организация работ 11

1.4 Используемые методы, подходы и инструменты 13

1.5 Разработка клиентской части на React 15

1.6 Реализованные экраны и компоненты интерфейса 17

1.7 Хранение данных и архитектура системы 20

2 РЕЗУЛЬТАТЫ ПРАКТИКИ 22

2.1 Реализованная структура системы Memory Hub 22

2.2 Основные технические решения 24

2.3 Проверка работоспособности системы 26

2.4 Соответствие результатов индивидуальному заданию 28

ЗАКЛЮЧЕНИЕ 30

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 31

ПРИЛОЖЕНИЕ А. ДИАГРАММА ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ СИСТЕМЫ MEMORY HUB 34

ПРИЛОЖЕНИЕ Б. СКРИНШОТЫ ИНТЕРФЕЙСА СИСТЕМЫ MEMORY HUB 35

ВВЕДЕНИЕ

Преддипломная практика является завершающим этапом подготовки к защите выпускной квалификационной работы. В рамках данного этапа выполняется практическая работа, направленная на получение результата, который может быть использован при дальнейшем развитии проекта и его представлении на защите. В ходе практики были закреплены навыки, необходимые при разработке современных веб-приложений: анализ предметной области, проектирование архитектуры системы, реализация клиентской части приложения и проверка её работоспособности.

Тематика практики связана с разработкой интеллектуальной системы личной памяти — Memory Hub. Концепция проекта строится на следующей идее: современный человек ежедневно сталкивается с огромным потоком информации — заметками, ссылками, идеями, задачами и файлами. При этом традиционные инструменты хранения данных (блокноты, папки, облачные хранилища) не обеспечивают ни смысловой классификации, ни семантического поиска, ни выявления связей между записями. Memory Hub призван стать единым пространством для хранения личной информации, где данные автоматически классифицируются, связываются между собой и могут быть найдены по смыслу, а не только по точному ключевому слову.

Объектом практики является процесс проектирования и разработки клиентской части веб-приложения Memory Hub — то, как пользователь взаимодействует с системой, какие экраны он видит и как он проходит основные пользовательские сценарии. Предметом практики являются конкретные технические и архитектурные решения: структура компонентов, логика маршрутизации, хранение данных в LocalStorage, реализация

основных функций (добавление, поиск, классификация, связи) и пользовательский интерфейс.

Цель практики — разработка работающего прототипа системы Memory Hub, покрывающего основные пользовательские сценарии. Для достижения этой цели были поставлены следующие задачи:

а) провести анализ предметной области и изучить существующие аналоги систем управления знаниями;

б) сформулировать функциональные требования к системе и выделить ключевые пользовательские сценарии;

в) спроектировать архитектуру клиентской части на основе React + Vite;

г) реализовать основные модули интерфейса: главный экран, библиотека, шкала времени, поиск, чат, граф связей, настройки;

д) провести ручное тестирование системы и проверить соответствие результатов индивидуальному заданию.

Практика проходила в период с 09.02.2026 по 05.04.2026. За это время клиентская часть системы была доведена до состояния, пригодного для демонстрации: реализованы все основные экраны, пользовательский путь проверен от начала до конца, а результаты подготовлены для представления на предзащите.

1 ОПИСАНИЕ РАБОТЫ, ВЫПОЛНЕННОЙ В ПЕРИОД ПРАКТИКИ

1.1 Характеристика предметной области и постановка задачи

Управление персональными знаниями (Personal Knowledge Management, PKM) — это практика сбора, организации, хранения и извлечения информации, которую человек накапливает в процессе учёбы, работы и повседневной жизни. В эпоху информационной перегрузки эта задача становится особенно актуальной: по различным оценкам, современный человек ежедневно сталкивается с объёмом информации, сопоставимым с содержанием нескольких книг. При этом большая часть этой информации теряется, не будучи зафиксирована, или оказывается недоступна в нужный момент из-за отсутствия удобного инструмента поиска.

Существующие решения в этой области можно разделить на три основные категории. К первой относятся простые заметочники — такие как Notion [3] и Obsidian [4]. Они предоставляют пользователю возможность создавать текстовые записи и организовывать их в папки или теги, однако не обеспечивают автоматической классификации и не выявляют семантических связей между записями. Ко второй категории относятся системы управления задачами (Todoist, Trello), ориентированные на планирование, а не на накопление знаний. Третья категория — специализированные системы управления знаниями, такие как Roam Research и Logseq. Они поддерживают граф связей между записями, однако обладают высоким порогом вхождения и требуют значительных усилий по поддержанию структуры со стороны пользователя.

Memory Hub отличается от перечисленных аналогов тем, что стремится минимизировать ручную работу пользователя по организации данных:

система автоматически предлагает теги на основе анализа текста, выявляет смысловые связи между записями и позволяет осуществлять семантический поиск — то есть поиск по смыслу, а не только по точному вхождению ключевых слов. При этом система поддерживает несколько типов контента: текстовые заметки, ссылки, файлы и задачи, что делает её универсальным инструментом для разных сценариев использования.

Постановка задачи для данной практики была сформулирована следующим образом: необходимо разработать клиентскую часть системы Memory Hub, реализующую основной пользовательский путь от добавления записи до её поиска и просмотра связанных воспоминаний. На уровне данной практики архитектурная граница была определена следующим образом: реализуется интерфейс и логика работы с данными на стороне клиента с использованием LocalStorage в качестве хранилища; серверная часть, база данных и NLP-модули выносятся за рамки MVP и будут подключены на следующем этапе разработки при написании ВКР.

Для уточнения требований к системе был выполнен сравнительный анализ аналогов, результаты которого представлены в таблице 1.1.

Таблица 1.1 — Сравнение Memory Hub с системами-аналогами

Критерий	**Notion**	**Obsidian**	**Roam Research**	**Memory Hub**
Автоматическая классификация	–	–	–	+
Семантический поиск	–	–	–	+
Граф связей между записями	–	+	+	+

Низкий порог вхождения	+	–	–	+
AI-ассистент для работы с памятью	–	–	–	+
Шкала времени	–	–	–	+

На основании сравнения сделан вывод, что Memory Hub занимает нишу, которую не закрывает ни один из существующих инструментов: система сочетает в себе низкий порог вхождения, автоматическую классификацию и семантический поиск. Это подтверждает актуальность разработки и её практическую значимость.

1.2 Функциональные требования и пользовательские сценарии

Перед началом проектирования интерфейса были сформулированы функциональные требования к системе. Основной пользователь Memory Hub — это студент, исследователь или специалист, который регулярно сталкивается с большим объёмом информации и испытывает потребность в её структурированном хранении и быстром доступе. Ключевое требование с точки зрения пользователя состоит в следующем: система должна принимать информацию с минимальными усилиями со стороны пользователя, самостоятельно обрабатывать её и предоставлять удобный доступ в нужный момент.

Исходя из этого, были определены следующие функциональные требования к системе:

а) пользователь должен иметь возможность быстро добавлять запись любого типа (текст, ссылка, файл, задача) через единую форму добавления, доступную с главного экрана;

б) система должна автоматически предлагать теги на основе содержимого записи, а пользователь должен иметь возможность принять или изменить предложенные теги;

в) все записи должны быть доступны в библиотеке с возможностью фильтрации по типу, тегу и дате;

г) система должна отображать записи в хронологическом порядке на шкале времени;

д) поиск должен работать по смыслу, а не только по точному совпадению слов;

е) система должна предоставлять AI-ассистента, способного отвечать на вопросы пользователя на основе накопленных записей;

ж) система должна визуализировать связи между записями в виде интерактивного графа;

з) пользователь должен иметь возможность настраивать параметры системы (язык, тема оформления).

На основании требований были выделены ключевые пользовательские сценарии. Основной сценарий — добавление и поиск записи — выглядит следующим образом: пользователь входит в систему, вводит заголовок и содержимое записи на главном экране или через кнопку «Добавить», система предлагает теги, пользователь сохраняет запись и впоследствии находит её через поиск или шкалу времени. Дополнительный сценарий — работа с AI-ассистентом: пользователь задаёт вопрос в разделе «Чат», система формирует ответ на основе накопленных записей. Третий сценарий — исследование связей: пользователь открывает раздел «Связи» и видит граф, на котором отображены смысловые связи между его записями, что позволяет обнаруживать неочевидные взаимосвязи между накопленными знаниями.

Ключевые функции системы были зафиксированы через диаграмму вариантов использования (Use Case), представленную в Приложении А. Эта диаграмма используется как рабочий ориентир: по ней проверяется, что каждый важный сценарий имеет соответствующий экран в интерфейсе и что пользователь может пройти полный путь без логических пробелов.

В системе выделена одна роль пользователя — Владелец памяти. Это намеренное упрощение: в отличие от платформ с множеством ролей, Memory Hub является персональным инструментом и не предполагает разделения прав доступа на уровне MVP. В дальнейшем возможно расширение функциональности за счёт добавления коллаборативных функций, однако для данного этапа архитектура рассчитана на одного пользователя. В таблице 1.2 приведён перечень функций системы с распределением по приоритетам.

Таблица 1.2 — Функциональные требования к системе Memory Hub

Функция	**Приоритет**	**Статус реализации**
Добавление записи	Высокий	Реализовано
Просмотр библиотеки	Высокий	Реализовано
Шкала времени	Высокий	Реализовано
Семантический поиск	Высокий	Реализовано (Mock)
AI-чат (ассистент)	Средний	Реализовано (Mock)
Граф связей	Средний	Реализовано
Автоматическая классификация	Средний	Заложено в архитектуру
Настройки системы	Низкий	Реализовано

1.3 Календарный план и организация работ

Работа в период практики была организована по этапам, поскольку отсутствие чёткой структуры разработки может привести к несогласованности отдельных компонентов системы и увеличению времени выполнения задач. При организации работ был использован подход, предполагающий последовательный переход от общего представления о системе к проектированию конкретных экранов и компонентов. В связи с этим работа была разделена на три логических этапа, отражённых в календарном плане (таблица 1.3).

Таблица 1.3 — Календарный план прохождения практики

Этап	Содержание работ	Сроки	Результат
Организационный	Анализ предметной области, изучение аналогов, формирование требований и архитектуры системы	09.02.2026 – 15.02.2026	Сформирован список требований и структура экранов
Основной	Разработка компонентов интерфейса, реализация основных модулей системы, настройка маршрутизации и хранения данных	16.02.2026 – 29.03.2026	Реализованы все основные экраны и модули системы
Заключительный	Тестирование пользовательских	30.03.2026 – 05.04.2026	Готов работающий

	сценариев, исправление ошибок, подготовка материалов для отчёта		прототип и материалы для защиты

Организационный этап занял период с 09.02.2026 по 15.02.2026. В рамках этого этапа было детально проанализировано задание на практику, уточнена граница разработки (что входит в MVP данной практики, а что выносится на этап написания ВКР), а также изучены существующие системы управления знаниями — Notion, Obsidian и Roam Research. Анализ аналогов позволил сформулировать ключевые отличия Memory Hub и определить, какие функции необходимо реализовать в первую очередь. На основании этого анализа была составлена структура экранов системы и определён стек технологий.

Основной этап занял наибольшую часть времени — с 16.02.2026 по 29.03.2026. На этом этапе была реализована клиентская часть системы. Разработка велась итеративно: сначала была настроена базовая структура проекта (React + Vite, маршрутизация, система хранения данных в LocalStorage), затем последовательно реализовывались модули — главный экран с дашбордом, библиотека, шкала времени, поиск, AI-чат, граф связей и настройки. На каждой итерации проверялась работоспособность реализованного функционала, после чего вносились корректировки.

Заключительный этап выполнялся с 30.03.2026 по 05.04.2026. На этом этапе основной задачей было не добавление новых функций, а доведение системы до состояния, пригодного для демонстрации. Были проверены все основные пользовательские сценарии: добавление записи, просмотр в библиотеке и на шкале времени, поиск, взаимодействие с AI-ассистентом, просмотр графа связей. По итогам проверки были исправлены несколько

ошибок в логике навигации и устранены визуальные недостатки. Также на этом этапе были подготовлены материалы для отчёта.

1.4 Используемые методы, подходы и инструменты

Выбор технологического стека был обусловлен несколькими соображениями. Во-первых, стек должен был обеспечить быструю разработку прототипа без избыточной сложности. Во-вторых, выбранные технологии должны быть актуальны с точки зрения современной frontend-разработки. В-третьих, стек должен был допускать дальнейшее масштабирование — подключение серверной части и NLP-модулей на следующем этапе работы.

Основным языком разработки был выбран JavaScript (без TypeScript) — это обеспечило скорость разработки и снизило накладные расходы на типизацию при работе над MVP. В качестве библиотеки для построения пользовательского интерфейса использован React [5] — стандарт де-факто в современной frontend-разработке, обеспечивающий компонентный подход и эффективное управление состоянием. Сборка проекта выполняется с помощью Vite [6] — современного инструмента сборки, обеспечивающего мгновенный старт сервера разработки и быструю пересборку при изменениях.

Для стилизации компонентов использован Tailwind CSS [7] — утилитарный CSS-фреймворк, позволяющий описывать стили непосредственно в JSX-разметке. Этот подход ускоряет разработку, поскольку устраняет необходимость переключения между файлами компонентов и файлами стилей. Маршрутизация в приложении реализована с помощью React Router [8], что позволяет строить одностраничное приложение (SPA) с несколькими логически изолированными экранами.

В качестве хранилища данных на уровне MVP используется LocalStorage браузера. Это решение продиктовано следующими соображениями: на данном этапе не требуется серверная инфраструктура, данные хранятся локально и доступны без подключения к сети, а сама логика работы с данными сформирована таким образом, что при необходимости LocalStorage может быть заменён на API-вызовы к серверу без изменения компонентов. Разработка велась в облачной среде Replit [9], что обеспечивало доступность проекта с любого устройства и упрощало демонстрацию результатов.

Методологически разработка велась итеративно: каждый модуль системы сначала проектировался на уровне структуры компонентов и логики взаимодействия с данными, затем реализовывался и проверялся вручную. После проверки при необходимости вносились корректировки, и работа переходила к следующему модулю. Такой подход позволяет на каждом шаге видеть работающий результат и не накапливать ошибки.

1.5 Разработка клиентской части на React

Главная идея архитектуры клиентской части Memory Hub была сформулирована следующим образом: система должна ощущаться как единое связное пространство, а не как набор разрозненных экранов. Пользователь должен иметь возможность быстро перемещаться между разделами, не теряя контекста, и понимать в любой момент, где он находится и что он может сделать дальше. Для достижения этого была выбрана классическая компоновка SPA с постоянной навигационной панелью слева и рабочей областью справа.

Структура проекта организована по принципу разделения ответственности. Корневой компонент App.jsx отвечает за инициализацию маршрутизации и передачу глобального состояния через React Context. Компонент Layout.jsx реализует общую раскладку страницы — боковую панель навигации и основную рабочую область. Каждый раздел системы реализован как отдельный компонент-страница (Dashboard, Library, Timeline, Search, Chat, Graph, Settings), что обеспечивает изоляцию логики и упрощает дальнейшее сопровождение кода.

Для управления данными о воспоминаниях был разработан пользовательский хук useMemories, инкапсулирующий логику чтения, записи и обновления данных в LocalStorage. Такой подход позволяет всем компонентам-страницам работать с данными через единый интерфейс, не дублируя логику взаимодействия с хранилищем. При замене LocalStorage на API-вызовы достаточно будет изменить только этот хук, не затрагивая остальные компоненты.

Боковая навигационная панель содержит следующие разделы: Главная, Библиотека, Шкала времени, Поиск, Чат, Связи и Настройки. Панель постоянно видна пользователю и обеспечивает быстрый переход между разделами в один клик. В нижней части панели отображается имя текущего пользователя и кнопка выхода, что создаёт ощущение персонального пространства. Вверху панели расположена кнопка «+ Добавить» — основное действие в системе — что соответствует принципу «самое важное действие должно быть в самом доступном месте».

Цветовая схема системы построена на фиолетово-белой палитре: основной акцентный цвет — фиолетовый (#7C3AED), фон рабочей области — светло-серый (#F9FAFB), карточки воспоминаний — белые с мягкой тенью. Такое решение обеспечивает высокий контраст и читаемость при сохранении визуальной привлекательности. Интерфейс не перегружен элементами: на каждом экране чётко выделено одно основное действие и сопутствующий контент.

1.6 Реализованные экраны и компоненты интерфейса

В рамках практики были реализованы все основные экраны системы Memory Hub, покрывающие полный пользовательский путь от добавления записи до её поиска и просмотра связей. Ниже приводится описание каждого реализованного экрана.

Главный экран (Dashboard) является точкой входа пользователя в систему. Он содержит сводную статистику: общее количество воспоминаний, количество записей, добавленных на текущей неделе, и наиболее часто используемый тег. Ниже статистики отображается лента последних воспоминаний с возможностью перейти к полному списку. Справа от ленты расположена форма быстрого добавления — заголовок и содержимое записи можно ввести непосредственно с главного экрана, не переходя на отдельную страницу. Также справа отображаются популярные теги и распределение записей по типам (текст, ссылка и другие). Такая компоновка позволяет пользователю получить полное представление о своей «памяти» с одного взгляда.

Библиотека (Library) представляет собой полный список всех воспоминаний пользователя с возможностью фильтрации и сортировки. Каждое воспоминание отображается в виде карточки, содержащей заголовок, краткое содержание, теги, тип и дату добавления. Пользователь может отфильтровать список по типу контента или по тегу, а также изменить порядок сортировки. Карточки оформлены компактно: при большом количестве записей пользователь может быстро просканировать список и найти нужное.

Шкала времени (Timeline) отображает воспоминания в хронологическом порядке, группируя их по датам. Этот экран позволяет пользователю «прокрутить» историю своих записей и найти нужную по времени добавления. Хронологическое представление особенно полезно для восстановления контекста: пользователь может вспомнить, что именно он изучал или на чём работал в определённый период.

Поиск (Search) реализует функцию семантического поиска по всем воспоминаниям пользователя. На уровне MVP поиск работает по совпадению ключевых слов в заголовке и содержимом записи, однако архитектура компонента предусматривает подключение векторного поиска на следующем этапе без изменения интерфейса. Результаты поиска отображаются в том же формате карточек, что и в библиотеке, что обеспечивает консистентность интерфейса.

AI-чат (Chat) предоставляет пользователю возможность взаимодействовать с системой в диалоговом режиме. Пользователь задаёт вопрос, и система формирует ответ на основе накопленных воспоминаний. На уровне MVP реализован Mock-режим: ответы генерируются по шаблонам, имитирующим работу AI-ассистента. Этот подход позволяет продемонстрировать концепцию и ценность функции, не требуя подключения реального языкового API на данном этапе. Интерфейс чата стандартный: слева — сообщения пользователя, справа — ответы системы, внизу — поле ввода.

Граф связей (Graph) визуализирует смысловые отношения между воспоминаниями пользователя в виде интерактивной сети узлов и рёбер. Каждый узел представляет одно воспоминание, рёбра — связи между записями, имеющими общие теги или смысловые пересечения. Пользователь может взаимодействовать с графом: перемещать узлы, нажимать на них для

просмотра деталей записи. Этот экран реализует одну из ключевых концепций Memory Hub — «карту личной памяти».

Настройки (Settings) позволяют пользователю управлять параметрами системы: выбирать язык интерфейса, переключаться между светлой и тёмной темами, а также управлять данными (экспорт, очистка). Раздел оформлен как стандартная страница настроек с группировкой параметров по тематическим блокам.

В таблице 1.4 приведён полный перечень реализованных экранов с описанием их назначения.

Таблица 1.4 — Реализованные экраны системы Memory Hub

Экран	**Назначение**
Главная (Dashboard)	Сводная статистика, последние записи, быстрое добавление
Библиотека (Library)	Полный список воспоминаний с фильтрацией и сортировкой
Шкала времени (Timeline)	Хронологическое представление записей по датам
Поиск (Search)	Семантический поиск по содержимому всех записей
Чат (Chat)	AI-ассистент для работы с накопленными знаниями
Граф связей (Graph)	Интерактивная визуализация связей между записями
Настройки (Settings)	Управление параметрами системы и данными пользователя

1.7 Хранение данных и архитектура системы

Архитектура клиентской части системы Memory Hub построена по принципу однонаправленного потока данных, характерного для React-приложений. Данные о воспоминаниях хранятся в LocalStorage в виде JSON-массива объектов. Каждый объект воспоминания имеет следующую структуру: уникальный идентификатор (UUID), заголовок, содержимое, тип (text, link, file, task), массив тегов, дата и время создания, а также массив идентификаторов связанных воспоминаний.

Глобальное состояние приложения управляется через React Context. Контекст предоставляет всем компонентам доступ к следующим данным и функциям: текущий массив воспоминаний, функции добавления, обновления и удаления воспоминаний, а также параметры пользователя (имя, настройки темы и языка). Такой подход позволяет избежать prop drilling — передачи данных через несколько промежуточных компонентов — и обеспечивает чистоту архитектуры.

Синхронизация между состоянием приложения и LocalStorage реализована через useEffect-хуки: при каждом изменении массива воспоминаний данные автоматически сериализуются в JSON и записываются в LocalStorage. При инициализации приложения данные считываются из LocalStorage и загружаются в состояние. Это обеспечивает персистентность данных между сессиями без необходимости серверного хранилища.

Архитектура была спроектирована с учётом будущего масштабирования. Слой работы с данными (хуки useMemories и useSettings) полностью изолирован от слоя представления (компоненты-страницы). Это означает, что при переходе от LocalStorage к серверному API достаточно

будет заменить реализацию хуков, не затрагивая компоненты интерфейса. Аналогично, подключение реального NLP-сервиса для автоматической классификации и семантического поиска потребует изменений только в соответствующих сервисных функциях, но не в компонентах, которые их вызывают. Такое разделение обеспечивает высокую сопровождаемость кода и упрощает дальнейшее развитие системы в рамках ВКР.

2 РЕЗУЛЬТАТЫ ПРАКТИКИ

2.1 Реализованная структура системы Memory Hub

Главным результатом практики является работающий прототип системы Memory Hub, реализованный в виде React-приложения и развёрнутый в среде Replit. Это не просто набор статичных макетов, а полноценное веб-приложение, в котором пользователь может добавлять воспоминания, просматривать их в различных форматах, искать нужные записи и взаимодействовать с AI-ассистентом. Данные сохраняются между сессиями благодаря LocalStorage, что делает систему пригодной для реальной эксплуатации уже на уровне MVP.

Реализованная структура системы охватывает семь функциональных модулей: главный экран (Dashboard), библиотека (Library), шкала времени (Timeline), поиск (Search), AI-чат (Chat), граф связей (Graph) и настройки (Settings). Каждый модуль реализован в виде отдельного компонента-страницы с собственной логикой, подключённого к глобальному состоянию через React Context.

Ниже приведена сводная оценка степени реализации функциональных требований, сформулированных на начальном этапе практики. Все требования с высоким приоритетом реализованы в полном объёме. Требования со средним приоритетом реализованы либо в полном объёме (граф связей), либо в режиме Mock с сохранением правильного пользовательского сценария (AI-чат, семантический поиск). Требование по автоматической классификации заложено в архитектуру — структура данных и интерфейс поддерживают теги и их автоматическое присвоение, однако

реальный NLP-модуль будет подключён на следующем этапе при написании ВКР.

Система готова к демонстрации: по ней можно провести полный пользовательский путь — от добавления первой записи до просмотра графа связей — без логических пробелов и ошибок в навигации. Это позволяет использовать прототип как «источник правды» при дальнейшей разработке серверной части и NLP-модулей в рамках ВКР.

2.2 Основные технические решения

В ходе разработки был принят ряд технических решений, которые оказали существенное влияние на архитектуру системы и определили её дальнейшее развитие.

Первое ключевое решение — использование LocalStorage в качестве хранилища данных для MVP. Это решение было принято осознанно: оно позволяет сосредоточить усилия на реализации пользовательского интерфейса и логики работы с данными, не отвлекаясь на разворачивание серверной инфраструктуры. При этом слой работы с данными спроектирован так, что замена LocalStorage на REST API или GraphQL-клиент потребует минимальных изменений в кодовой базе.

Второе решение — отказ от TypeScript в пользу JavaScript на уровне MVP. TypeScript обеспечивает более строгую типизацию и лучшую поддержку со стороны IDE, однако добавляет накладные расходы на описание типов и конфигурацию компилятора. Для быстрой разработки прототипа JavaScript оказался более подходящим выбором. При переходе к финальной версии системы в рамках ВКР предполагается миграция на TypeScript.

Третье решение — реализация AI-чата и семантического поиска в режиме Mock на уровне MVP. Это решение позволяет продемонстрировать концепцию и ценность функций, не требуя подключения реального языкового API или развёртывания NLP-сервиса. Mock-режим реализован таким образом, что пользовательский сценарий в нём полностью совпадает со сценарием работы с реальным AI-ассистентом — это важно для демонстрации и для дальнейшей разработки.

Четвёртое решение — использование React Context для управления глобальным состоянием вместо внешней библиотеки (Redux, Zustand). Для масштаба текущего приложения встроенных средств React достаточно, и их использование упрощает архитектуру. При значительном росте сложности состояния в будущем возможна миграция на специализированную библиотеку.

2.3 Проверка работоспособности системы

Проверка работоспособности системы проводилась вручную путём прохождения всех ключевых пользовательских сценариев. На данном этапе разработки в качестве основного метода проверки было выбрано ручное тестирование — оно позволяет оценить не только корректность работы функций, но и качество пользовательского опыта — насколько интуитивен интерфейс, нет ли неочевидных мест, где пользователь может остановиться.

В первую очередь был проверен базовый сценарий добавления записи. Проверялось, насколько быстро можно найти форму добавления, понятно ли, какие поля обязательны, и корректно ли запись появляется в списке после сохранения. По результатам проверки было принято решение разместить форму быстрого добавления непосредственно на главном экране, а не только на отдельной странице — это сокращает количество кликов и делает сценарий добавления максимально быстрым.

Затем был проверен сценарий поиска. Были введены несколько поисковых запросов, в том числе запросы, частично совпадающие с содержимым записей, и запросы, не дающие результатов. В последнем случае система отображает сообщение «Ничего не найдено», что важно для понимания пользователем результата поиска.

Был проверен AI-чат: пользователь задаёт вопрос, система формирует ответ. Проверялось, что интерфейс чата работает корректно: сообщения отображаются в правильном порядке, поле ввода очищается после отправки, ответ системы визуально отличается от сообщений пользователя.

Граф связей был проверен при наличии нескольких записей с пересекающимися тегами. Проверялось, что узлы и рёбра отображаются корректно, взаимодействие с узлами работает, и пользователь может получить доступ к деталям записи непосредственно из графа.

По результатам тестирования был составлен перечень правок, которые были внесены до завершения практики. В частности, была улучшена навигация между экранами: добавлены кнопки «назад» там, где они отсутствовали, и исправлено поведение активного пункта меню при переходе между разделами. Также были устранены несколько визуальных несоответствий в карточках воспоминаний.

2.4 Соответствие результатов индивидуальному заданию

В рамках индивидуального задания на практику необходимо было сосредоточиться на разработке клиентской части системы Memory Hub, реализующей основной пользовательский путь. По итогам практики все задачи, сформулированные во введении, выполнены в полном объёме.

Первая задача — провести анализ предметной области и изучить аналоги — выполнена: в разделе 1.1 представлен сравнительный анализ систем Notion, Obsidian и Roam Research, на основе которого были сформулированы ключевые отличия Memory Hub.

Вторая задача — сформулировать функциональные требования и пользовательские сценарии — выполнена: в разделе 1.2 представлен перечень функциональных требований и описаны три основных пользовательских сценария.

Третья задача — спроектировать архитектуру клиентской части — выполнена: в разделах 1.4–1.5 описаны выбранный технологический стек и архитектурные решения. В разделе 1.7 описана архитектура управления данными.

Четвёртая задача — реализовать основные модули интерфейса — выполнена: в разделе 1.6 описаны все семь реализованных экранов системы. Скриншоты интерфейса приведены в Приложении Б.

Пятая задача — провести тестирование — выполнена: в разделе 2.3 описан процесс ручного тестирования и результаты проверки основных пользовательских сценариев.

Итоговый результат практики — работающий прототип системы Memory Hub — является хорошей основой для следующего этапа: подключения серверной части, базы данных и NLP-модулей в рамках ВКР. Сейчас у проекта есть понятная структура компонентов, отлаженная логика пользовательских сценариев и прототип, который можно использовать как «источник правды» при реализации серверной части.

ЗАКЛЮЧЕНИЕ

В ходе преддипломной практики были выполнены работы по клиентской части системы Memory Hub, в результате чего она была доведена до состояния, достаточного для перехода к следующему этапу разработки. Результат практики — не просто набор описаний и диаграмм, а работающий программный прототип, по которому можно провести полный пользовательский путь от добавления первой записи до просмотра графа связей между воспоминаниями.

В начале работы была проведена исследовательская часть: изучена предметная область управления персональными знаниями, выполнен сравнительный анализ систем Notion, Obsidian и Roam Research. Анализ показал, что ни одна из существующих систем не сочетает в себе все три ключевые характеристики: низкий порог вхождения, автоматическую классификацию и семантический поиск. Именно эта ниша и определяет позиционирование Memory Hub.

На основании анализа были сформулированы функциональные требования и пользовательские сценарии, которые стали основой для проектирования интерфейса. Ключевые сценарии — добавление записи, поиск, взаимодействие с AI-ассистентом и просмотр графа связей — были реализованы и проверены в ходе ручного тестирования.

Разработка клиентской части велась на стеке React + Vite + Tailwind CSS с использованием LocalStorage в качестве хранилища данных. Архитектура приложения спроектирована с учётом дальнейшего масштабирования: слой работы с данными изолирован от слоя представления,

что позволяет безболезненно заменить LocalStorage на серверный API при написании ВКР.

Были реализованы семь основных модулей: Главная (Dashboard), Библиотека, Шкала времени, Поиск, AI-чат, Граф связей и Настройки. Все модули прошли ручное тестирование; по результатам тестирования были устранены ошибки в навигации и визуальные несоответствия.

Полученный результат может рассматриваться как хорошая основа для следующего этапа работ — подключения серверной части на Node.js, базы данных и NLP-модулей для автоматической классификации и семантического поиска. Готовый прототип удобно использовать при демонстрации на предзащите: можно показать не только идею проекта, но и то, как именно пользователь взаимодействует с системой на каждом шаге.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Алферьева Т. И., Васина В. Н., Шадрин Д. Б. Методические рекомендации для подготовки и оформления выпускных квалификационных работ : методические рекомендации для обучающихся всех форм обучения укрупненной группы направлений подготовки 09.00.00 «Информатика и вычислительная техника». — Екатеринбург : УрФУ, 2019. — 94 с. — Электрон. текст. ресурс (pdf).

2. Notion — инструмент для совместной работы с вики, документами и проектами : сайт. — URL: https://www.notion.so/ (дата обращения: 20.03.2026).

3. Obsidian — частное и гибкое хранилище для заметок : сайт. — URL: https://obsidian.md/ (дата обращения: 20.03.2026).

4. Roam Research — инструмент для создания сетевых заметок : сайт. — URL: https://roamresearch.com/ (дата обращения: 25.02.2026).

5. Logseq — A privacy-first, open-source knowledge base : сайт. — URL: https://logseq.com/ (дата обращения: 28.02.2026).

6. React. Documentation : сайт. — URL: https://react.dev/ (дата обращения: 20.03.2026).

7. Vite. Next Generation Frontend Tooling : сайт. — URL: https://vitejs.dev/ (дата обращения: 20.03.2026).

8. Tailwind CSS. A utility-first CSS framework : сайт. — URL: https://tailwindcss.com/ (дата обращения: 20.03.2026).

9. React Router. Client Side Routing for React : сайт. — URL: https://reactrouter.com/ (дата обращения: 19.03.2026).

10. Replit — облачная среда разработки : сайт. — URL: https://replit.com/ (дата обращения: 5.03.2026).

11. MDN Web Docs. Web Storage API : сайт. — URL: https://developer.mozilla.org/ru/docs/Web/API/Web_Storage_API (дата обращения: 18.03.2026).

12. MDN Web Docs. Window: localStorage property : сайт. — URL: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage (дата обращения: 19.03.2026).

13. MDN Web Docs. JavaScript Guide : сайт. — URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide (дата обращения: 25.03.2026).

14. MDN Web Docs. JSON : сайт. — URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON (дата обращения: 21.03.2026).

15. Forte T. Building a Second Brain: A Proven Method to Organize Your Digital Life and Unlock Your Creative Potential. — New York : Atria Books, 2022. — 272 p.

ПРИЛОЖЕНИЕ А. ДИАГРАММА ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ СИСТЕМЫ MEMORY HUB

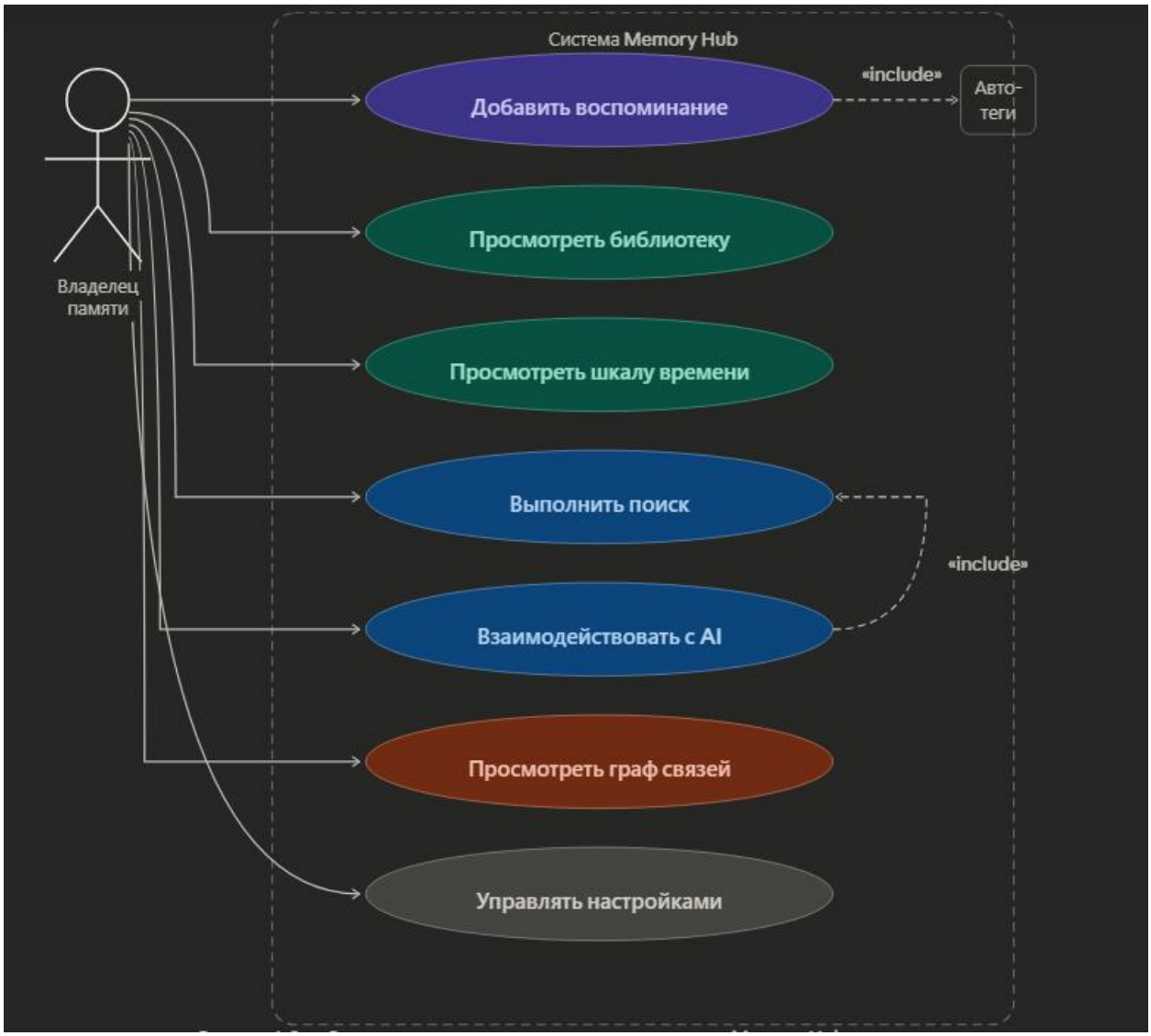

Рисунок А.1 — Диаграмма вариантов использования системы Memory Hub

ПРИЛОЖЕНИЕ Б. СКРИНШОТЫ ИНТЕРФЕЙСА СИСТЕМЫ MEMORY HUB

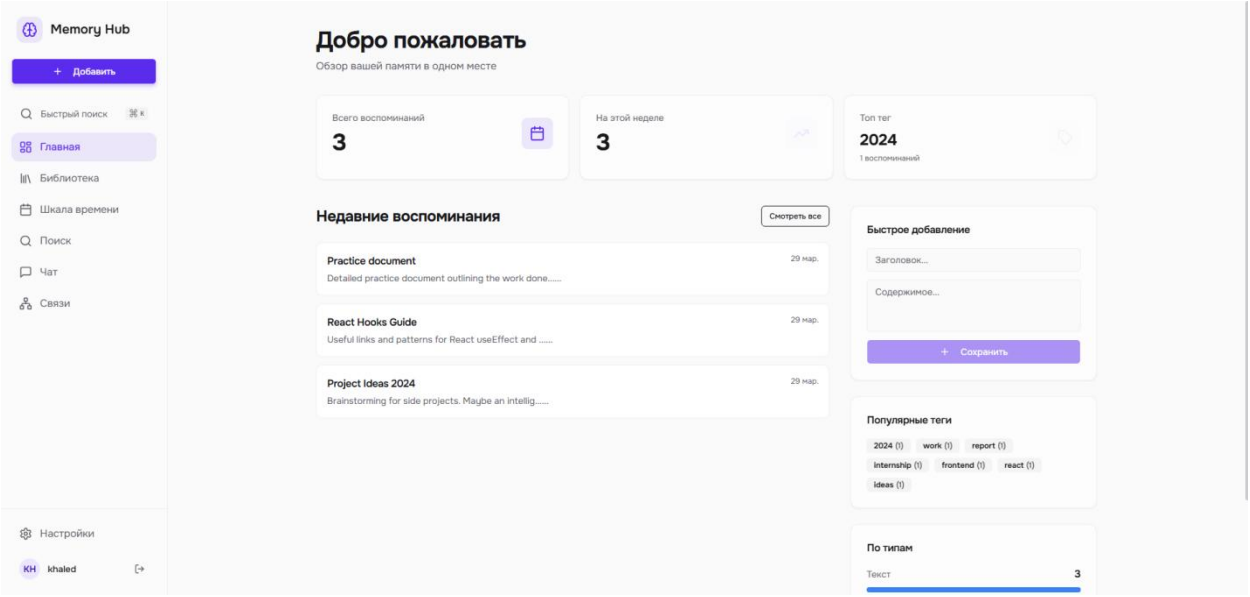

Рисунок Б.1 — Главный экран (Dashboard): статистика, последние воспоминания, форма быстрого добавления

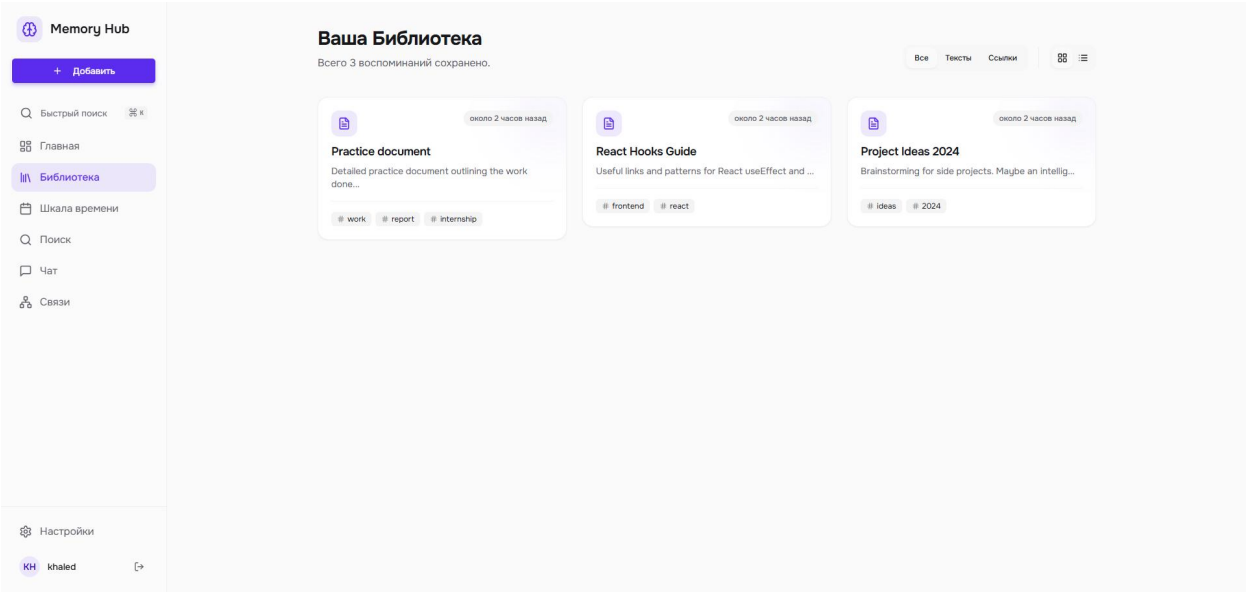

Рисунок Б.2 — Библиотека воспоминаний с карточками записей

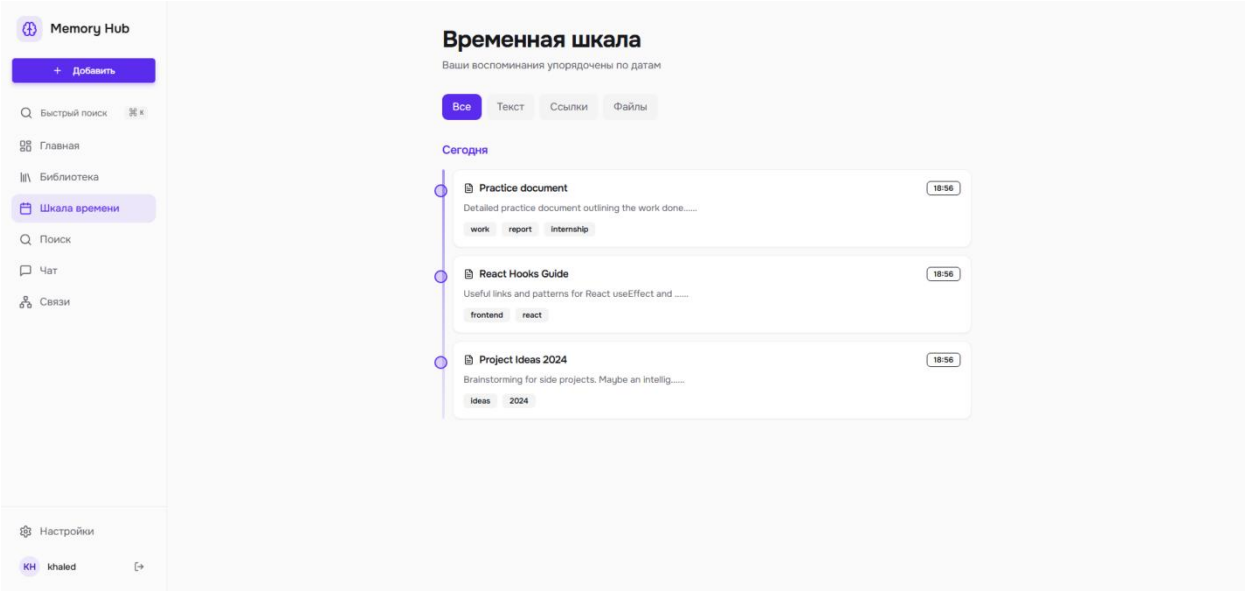

Рисунок Б.3 — Шкала времени с хронологическим отображением записей

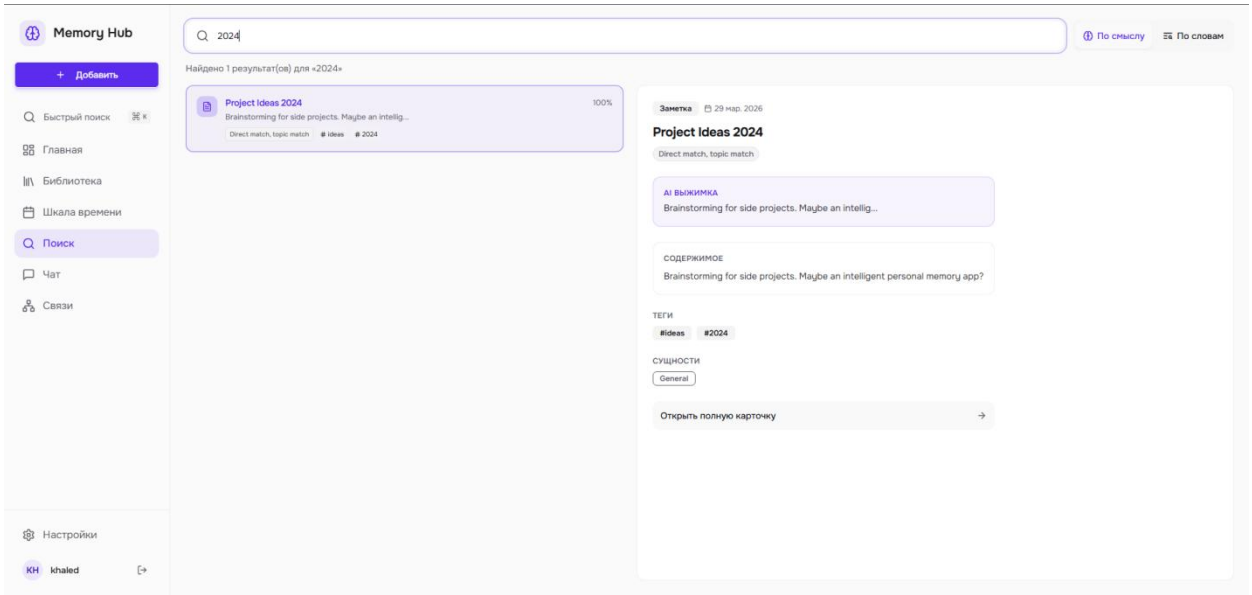

Рисунок Б.4 — Экран поиска

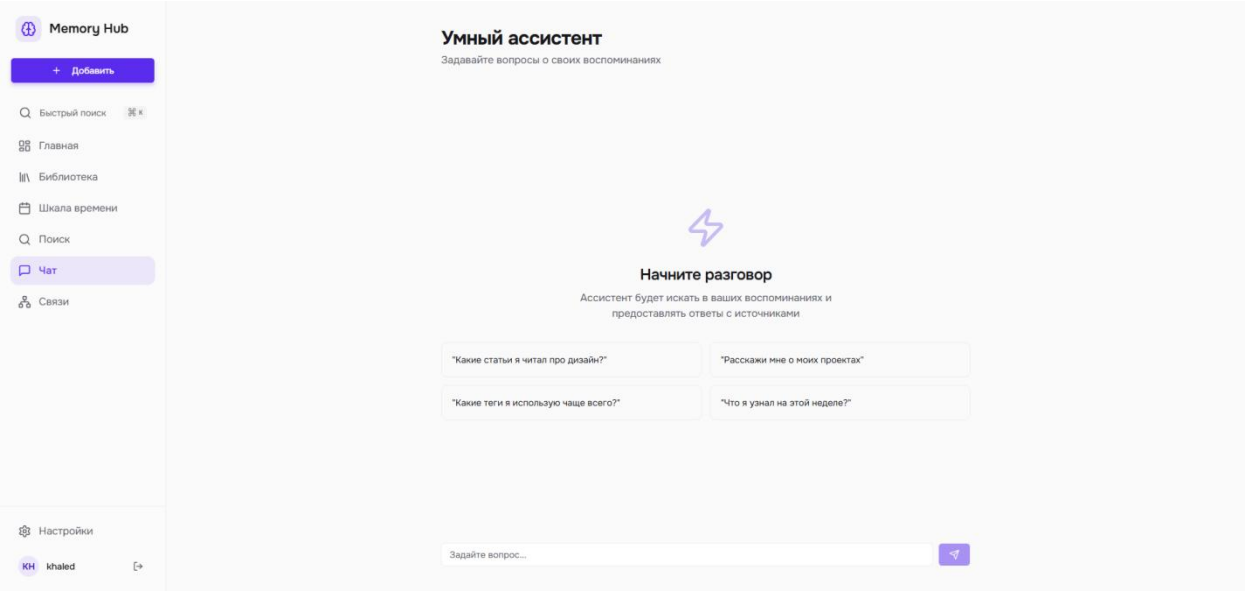

Рисунок Б.5 — Экран AI-ассистента

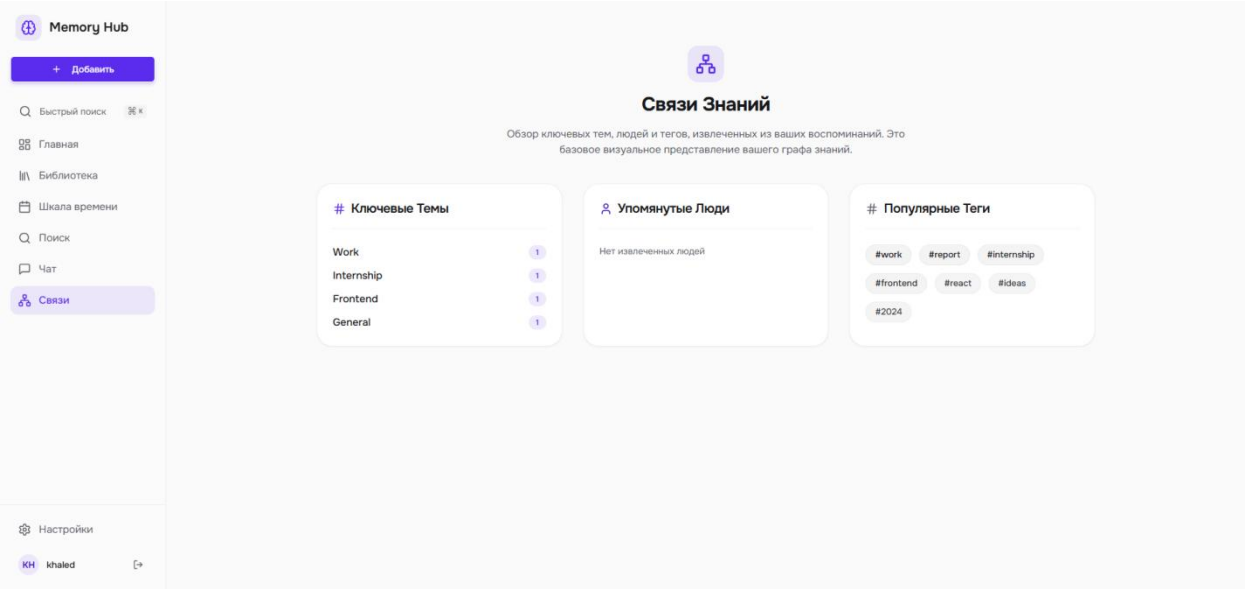

Рисунок Б.6 — Граф связей между воспоминаниями

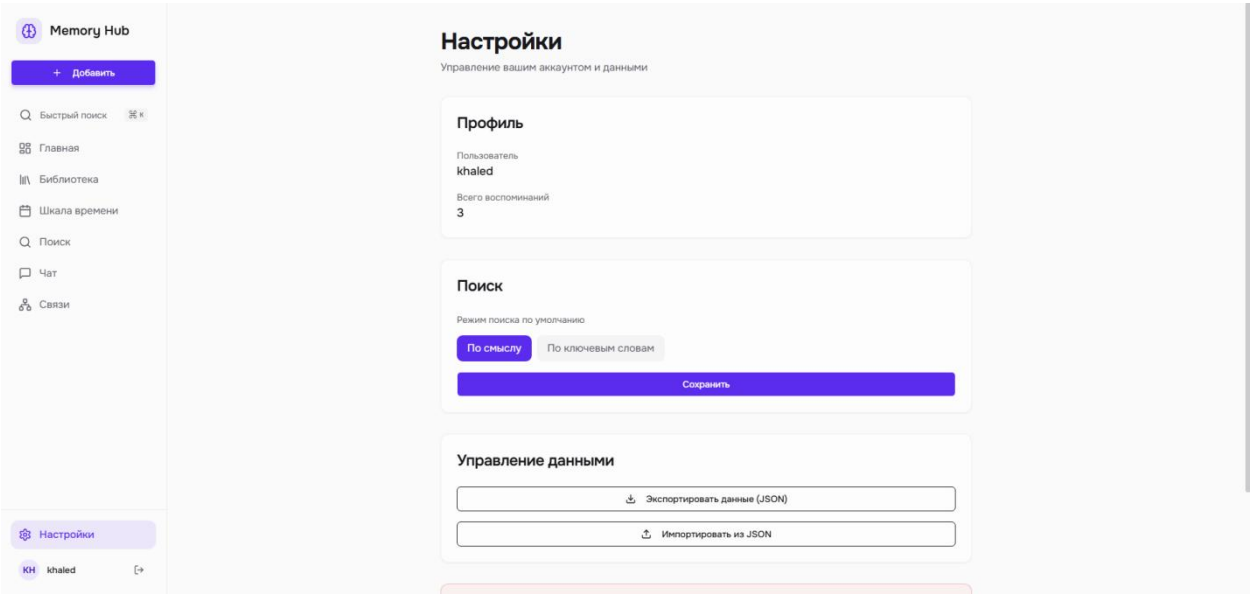

Рисунок Б.7 — Экран настроек системы